A Parametric Drought-Index Insurance Simulator for Smallholder Maize Farmers in Eastern Rwanda

Technical Report: Final Version of the Product

Jok John Maker Kur · BSc Software Engineering, African Leadership University · Supervisor: Neza David Tuyishimire · 29 October 2026

Links: Live dashboard https://rdi-maize-insurance.vercel.app · Live API https://rdi-drought-api.onrender.com/docs · Repository https://github.com/JokMaker/parametric-drought-simulator · USSD `*384*44698#` (Africa's Talking sandbox) · Walkthrough video: *[link]*

1. Overview

The product turns two satellite signals into a drought-insurance decision for Season A maize in Eastern Rwanda's seven districts. Rainfall (CHIRPS) and crop greenness (MODIS NDVI) are expressed as anomalies against each district's own history. A Ridge regression, calibrated on NISR survey yields, converts them into a predicted yield change (the index). One pooled payout rule then applies: payouts start at −2% (trigger) and rise linearly to a full payout at −22% (exit). Every payout is shown with its reason and its uncertainty. The gap between what the index pays and what farmers lost (basis risk) is reported for each district rather than hidden.

The system has three deployed parts: a **notebook** for data engineering and model training, a **FastAPI backend** (Python 3.12) that serves the model, stores farmer data and runs SMS and USSD, and a **Next.js 16 dashboard** of 12 pages for analysts. Farmers reach the product from any basic phone over USSD, in Kinyarwanda by default.

2. Scope Alignment and Feature Validation

The approved proposal (29 September 2026) set out ten functional requirements (FR1–FR10) and six non-functional requirements (NFR1–NFR6). The Phase column links each one to the proposal's Gantt chart (Section 1.10): 1 proposal and review, 2 data pipeline, 3 index construction, 4 calibration and model comparison, 5 dashboard and API, 6 backtesting, report and defence.

Planned feature	Phase	Intended purpose	Status	Evidence
FR1 Select a district and season, view drought risk	5	Analyst inspects any district-season	**Completed**	District Explorer `/explorer`; Drought Map `/map` (every season since 2001, Play button)
FR2 Retrieve CHIRPS, MODIS NDVI and NISR data	2	Ground the index in open data	**Completed (changed)**	`notebook/drought_index.ipynb` (Earth Engine extraction, NISR ingestion of 9 releases); exported to `backend/data/`
FR3 Rainfall-and-vegetation anomaly score against each district's baseline	3	Make districts comparable	**Completed**	z-scores in the notebook; served by `GET /api/simulate`; shown in the Explorer
FR4 Calibrate one pooled trigger and exit by regression	4	A single, explainable payout rule	**Completed**	Ridge vs Lasso vs quadratic OLS, nested LOOCV (notebook Sections 8–9); `model.json`: trigger −2%, exit −22%
FR5 Simulate a historical payout	3, 5	Show what the product would have paid	**Completed**	`IndexModel.payout()` in `backend/app/model.py`; Explorer payout gauge
FR6 Show the payout with its reason	5	Transparency for users	**Completed**	`reason()` in `frontend/lib/payout.ts`; Explorer and Farmer View
FR7 Show trigger and exit beside a district's anomalies	5	Show where a season sits on the rule	**Completed**	Explorer gauge between trigger and exit; Overview
FR8 Show which input drives the decision	4, 5	Explainability	**Completed**	Overview "What drives the index" (greenness carries 93% of the weight)
FR9 Backtest against documented drought years; compare with real yields	6	Credibility check	**Completed**	Drought History `/history`: payouts in 54% of drought-year seasons (34/63) vs 23% of other years (26/112). Explorer compares index and harvest for 2018–2025 (extended from 2022–2025)

Planned feature	Phase	Intended purpose	Status	Evidence
FR10 Report basis risk per district	6	Disclose the product's main weakness	**Completed**	Basis Risk `/basis-risk`: Nyagatare 33% (highest) to Kayonza 18% (lowest), plus a live trigger/exit simulator
NFR1 Fast response	5	Usable in normal conditions	**Met**	API median ≈1 ms; 1,960 requests/s with 100 clients; every page LCP < 2.5 s on a throttled phone (`docs/testing.md`)
NFR2 No personal data	5	Privacy (Law 058/2021)	**Partial (changed)**	Index data is district-level only. USSD sign-up stores phone numbers, only after the farmer confirms a screen stating "We keep your number for this cover"; numbers are masked in the UI and API, never in the repository, and only sandbox and test numbers are used (see below)
NFR3 Usable without statistics expertise	5	Programme officers can read results	**Met**	Plain-language card titles, guided tour on the Overview, reasons on every payout; Lighthouse accessibility 100
NFR4 Portable via Docker	5	Same behaviour everywhere	**Met**	`docker compose up --build`; the same Dockerfile runs on Render
NFR5 Reproducible thresholds	4	Auditable calibration	**Met**	Fixed inputs and seeds; tests check the API against the notebook's 175 index values
NFR6 Seven districts, Season A only	1	Keep scope feasible	**Met**	Scope unchanged

Added beyond the proposal, driven by supervisor feedback and the aim of a usable product: **This Season** (a live season monitor on NASA IMERG and MODIS daily data), **Pricing** (burning-cost premiums per district), **Field Checks** (a safety net for losses the index misses), **Farmer View** with an SMS through Africa's Talking, a **USSD menu** (cover, season, payouts, loss reports and **sign-up**, also playable on the dashboard's on-screen phone with a demo number), a **Policies** page, and Kinyarwanda as the default language.

Scope changes and why

- **Data: 21 → 56 district-seasons.** The proposal had three usable years. 2024 was rebuilt by joining raw Production and Screening files. On 16 October the supervisor approved the NISR 2017–2021 releases, so training now covers 2018–2025.
  - 2017 is kept but not trained on, because NISR imputed most of its production.
  - 2013–2016 have no harvest quantities in the microdata.
  - Yields are detrended (+9.3% a year) before computing anomalies.
  - An earlier R^2 of 0.51 on 2022–2025 turned out to be mostly that trend. The honest result on 56 seasons is a leave-one-out R^2 of 0.11, and it is reported as such.
- **Earth Engine on demand → precomputed export.** Calling Earth Engine per request would break NFR1 and require cloud credentials on the server. The notebook exports the results instead, and the season monitor refreshes them with a script.
- **PostgreSQL for computed results → files plus Postgres for user data.** The calibration output is small and read-only, so it ships as versioned files, which serves NFR5. Postgres (Neon) holds what users create: policies, loss reports and the SMS log.
- **Users: analysts → analysts and farmers.** The supervisor asked that Rwandans, the real users, meet Kinyarwanda first and a simple code. The proposal ruled out "real policyholder onboarding", so the sign-up is a **sandbox demonstration**: it runs only on Africa's Talking's sandbox and the dashboard's demo numbers (`+2507000…`), and no real farmer's data has been collected. The confirmation screen tells the farmer the number is kept before anything is saved. Before real use the project would need research-ethics approval and full compliance with Law 058/2021 on personal data (registration with the data-protection authority, written consent, retention limits and breach notification). This is the deviation from NFR2.
- **Not implemented, moved to future work:** an automatic payout run with mobile-money disbursement, an insurer login, early-warning SMS, live SMS with a registered sender ID (needs a TIN), and a dedicated short code (needs a RURA allocation).

3. Technical Implementation

Architecture.

- **Offline data:** the notebook pulls CHIRPS and MODIS through Google Earth Engine and NISR microdata. It writes `model.json`, `calibration.csv`, `seasons.csv`, `plot_losses.csv` and `current_season.json`.
- **Backend:** FastAPI loads these files once and computes index, payout, pricing and basis risk per request.

- **Dashboard:** Next.js fetches JSON from the backend. Its address is fixed at build time; with `NEXT_PUBLIC_USE_MOCK` the dashboard runs on saved results without a backend.
- **Africa's Talking:** calls `POST /api/ussd` on every key press, and the backend sends SMS through its messaging API.
- **Deployment:** the dashboard on Vercel; the API on Render (Docker, Frankfurt, described in `render.yaml`); the database on Neon Postgres, next to the API; UptimeRobot checks `/api/health` every 5 minutes, which also keeps the free server awake. Every push to `main` redeploys both services.

Key algorithms and technical logic

1. **Index and payout.**
   - Anomalies: `z = (x − mean_d) / sd_d` per district *d*, over October–December.
   - Index: `I = b0 + b1·ž_rain + b2·ž_ndvi`, on standardised inputs.
   - Payout share: `P = min(1, max(0, (T − I)/(T − E)))`, with T = –2 and E = –22.
2. **Nested leave-one-out cross-validation.**
   - Each of the 56 rows is held out once.
   - Ridge's α is tuned by an inner LOOCV on the remaining 55 only, so the held-out row never influences tuning.
   - Results, against a mean-only baseline:

Model	RMSE	Predictive R^2
Ridge (selected)	22.26	0.11
Lasso	22.13	0.13
Quadratic OLS	22.69	0.08
Mean-only baseline	24.09	–0.04

   - The proposal (Section 3.9) said the model with the lowest LOOCV error would be selected, which here is Lasso (RMSE 22.13 against 22.26). Ridge was kept as a deliberate deviation: the gap is 0.13 points, about 0.6% of the error, which is not a meaningful difference for an estimate from 56 rows, so the two are treated as tied. With rainfall and greenness correlated, Lasso can drop one of them entirely between refits, while Ridge keeps both with stable weights, which is easier to explain to farmers and insurers and steadier when new seasons are added.
3. **Trigger search.** Every whole-number trigger from –1% to –30% is scored on the leave-one-out index by hit rate minus false payout rate, and the best one is kept. The exit is set 20 points below it. That gives –2% / –22%: a hit rate of 50% (poor harvests paid) against a false payout rate of 22%, the trade-off research question 2 asked about.
4. **Uncertainty.** The leave-one-out RMSE (22.3 points) is treated as a normal error. The chance that the true harvest fell past the trigger is `Φ((T − I)/22.3)`, using a rational approximation of the normal CDF.
5. **Pricing.**
   - Value = area × expected yield × farm-gate price (RWF 200/kg).
   - Cover = value × 22%.
   - Premium = cover × the district's average payout over 25 seasons (burning cost) × (1 + loading 30%) × (1 − subsidy 40%).
   - The same calculation (`CoverQuote` in Python, `lib/pricing.ts` in TypeScript) is cross-checked by a test: both quote RWF 584 for policy RDI-RWA-1233.
6. **Field-check rule.** A district needs a field check when the payout is below 25% but there are at least 3 farmer reports, or at least 45% of NISR-surveyed farms harvested under half the expected yield. Season A 2019 flags all seven districts.
7. **Stateless USSD state machine.** Africa's Talking sends the whole session as `text`, for example `5*1*3*1250`. `Session.replay()` rebuilds the language and screen path from it:
   - `5` on the main menu switches language;
   - `0` pops a screen;
   - sign-up then reads the remaining answers in order and skips invalid ones, so a wrong district or field size repeats the question instead of ending the session. The server keeps no session state.
8. **SMS length.**
   - `sms_parts()` counts GSM-7 characters: 160 per SMS, 153 per part, extension characters count twice.
   - One non-GSM character switches the whole message to UCS-2 (70 per SMS).
   - Live mode refuses multi-part messages and stops at a daily cap.
9. **Phone matching.** Numbers are compared by their last nine digits, so `+250788…` and `0788…` are the same farmer. The database enforces one policy per number.
10. **Season-monitor reliability.** The provisional index is re-run on 25 past seasons at each date. It agrees with the final payout decision 60% of the time by mid-October and 78% at the close, and the dashboard shows that figure next to the provisional value.

Handling of inputs and errors

- **API:** unknown district → 404; invalid policy number or season → 422; SMS not configured → 503; provider rejection → 502 with the reason.
- **Dashboard:** keeps typed values while the user types and corrects them on blur (a field size of 10 is raised to the 50 m² minimum).
- **USSD:** invalid answers repeat the question.
- **Database:** an unreachable database fails after 8 s instead of 30, because Africa's Talking drops a USSD session at about 10 s.

4. Code Quality and OOP

The backend is split into modules with one job each: four routers (`index`, `farmers`, `policies`, `messaging`), `model.py` (`IndexModel`), `data.py` (`Dataset`), `policies.py` (`Policy`, `PolicyBook`, `CoverQuote`), `reports.py` (the store classes), `messaging.py` (the SMS gateways), `ussd.py` (`Session`, `UssdMenu`) and `dependencies.py`, which builds the objects and injects them into routes.

Principle	Where it shows
Encapsulation	Callers use `store.register()`, `dataset.result()` and `gateway.send()`, never SQL, file layouts or HTTP
Abstraction and polymorphism	`SmsGateway` (abstract) → `UnconfiguredGateway`, `SandboxGateway`, `LiveGateway`; `ReportStore` (abstract) → `SqliteStore`, `PostgresStore`. Callers do not know which one they hold
Inheritance	`AfricasTalkingGateway` holds the shared request. `LiveGateway` overrides `check()` and `fields()` to add the daily cap and sender ID. Every query is written once in `ReportStore`; subclasses only connect
Single responsibility	Each class has one reason to change: `IndexModel` the formula, `Dataset` the data files, `CoverQuote` the price, `UssdMenu` the screens, the stores the database
Open/closed and Liskov substitution	A new database or SMS provider is a new subclass; `SqliteStore` and `PostgresStore` pass the same parametrised tests, so either can replace the other without changing the routes
Dependency inversion	Routes receive the dataset, store, gateway and menu through FastAPI `Depends`; the gateway takes its HTTP transport as a parameter. Tests swap each one without touching real data
Immutable value objects	`SeasonResult`, `Policy`, `CoverQuote`, `SmsSettings` and `Session` are frozen dataclasses
Separation of concerns (frontend)	Calculations live in plain TypeScript modules under `lib/` (`payout`, `pricing`, `fieldChecks`, `sms`), apart from the components that display them. The UI is composed from reusable components such as `Card`, `StatStrip`, `NumberInput`, `DistrictMap` and the charts

Quality practices: the structure is documented in the README; names are consistent (`snake_case` in Python, `camelCase` in the JSON API through a shared `CamelModel`); every module has a docstring; secrets live only in environment variables; and the repository has 62 commits since 28 September, each describing one change.

5. Testing, Validation and Performance

Strategy	Why	How	Result
Unit and API tests (pytest)	The maths and endpoints must match the notebook exactly	68 tests, plus 2 that run against a throwaway Postgres schema; temporary database and a recording SMS gateway injected through dependency overrides	**70/70 pass** (SQLite and Postgres)
Frontend unit tests (Node test runner)	Pricing, payout, uncertainty and SMS logic run in the browser	33 tests, including a cross-check that the Pricing page and USSD quote the same premium	**33/33**
Functional browser tests	Real user flows with valid and invalid input	Headless Chrome via the DevTools protocol, 7 flows: policy lookup (lower case), unknown policy, area field (1 → 1250 → 10), SMS in two languages, USSD navigation, USSD sign-up with a wrong answer (district 9), pricing	**7/7**
Compatibility	Rwanda's phones run Chromium or WebKit	12 pages × phone 360 px / tablet / desktop × light and dark in Chrome 156; all pages at 3 widths in Firefox 159 (Gecko, geckodriver) and Safari 26 (WebKit, safaridriver)	**Chrome 72/72, Firefox 36/36, Safari 33/33**; Chrome and Firefox also run against the live site
Device testing	Farmers and field staff use budget Android phones	The live site checked by hand on a Tecno Camon Slim (Android, Chrome, Wi-Fi): Overview, Pricing, Farmer View, the USSD page, rotation; low-end phones also emulated (Slow 4G, CPU 4× slower)	All pages work and fit the screen; pages open in about 1–2 s on Wi-Fi

Strategy	Why	How	Result
Performance and load	USSD needs a reply within about 10 s; rural phones are slow	API benchmark and load test; pages on Slow 4G with the CPU slowed 4×; Lighthouse (mobile); live timings from Kigali	API median ≈1 ms; **1,960 req/s at 100 clients, 0 errors**; every page LCP ≤ 2.49 s throttled; Lighthouse Performance 93–97, Accessibility/Best Practices/SEO 100; live USSD screen 0.43 s
Model validation	The index must be honest about accuracy	Nested LOOCV, leave-one-season-out, drought-year backtest	R^2 0.11 (LOO), −0.11 for an unseen season; drought years paid 2.3× as often as other years

Improvements made because of testing:

- **Phone overflow.** The Overview scrolled sideways by 29 px on a 360 px phone. Chart labels now stack above the bars.
- **Season strip.** It lost the selected season after rotating a phone. It now re-centres on resize.
- **Contrast.** Lighthouse found text below WCAG AA (as low as 2.8:1). The colours were changed, and every text element now passes 4.5:1.
- **Deployment fixes:**
  - A setting pasted with a trailing line break broke the database connection; settings are now trimmed.
  - The uptime monitor's HEAD requests were refused; the health check now accepts them.
  - USSD SMS went to the demo phone instead of the caller; they now go to the caller.
  - A database outage could hold a USSD session for 30 s; it now fails after 8 s.

Limitations of the testing:

- The real-phone check was by hand on one phone over Wi-Fi; load times on a slow mobile connection were measured only in emulation.
- No usability sessions with farmers yet.
- The Kinyarwanda text is a draft awaiting a native speaker's review.

6. Conclusion and Next Steps

All ten functional requirements are implemented and deployed. Five of the six non-functional requirements are met, and the sixth (NFR2) changed for a justified reason. The proposal's research questions are answered honestly:

- The index has a measurable but weak relationship with yield (R^2 0.11).
- One pooled rule pays half of poor harvests, with 22% false payouts.
- Ridge and Lasso tie, and both beat quadratic OLS and the baseline.

Basis risk remains the product's main weakness, which is why it is disclosed per district and backed by field checks.

Next steps: ethics approval, a native-speaker review of the Kinyarwanda text, testing with farmers on their own phones, a payout run with mobile-money disbursement, live SMS with a registered sender ID, an insurer login, and re-calibration as new NISR seasons are released.

References

- Funk, C., Peterson, P., Landsfeld, M., Pedreros, D., Verdin, J., Shukla, S., Husak, G., Rowland, J., Harrison, L., Hoell, A., & Michaelsen, J. (2015). The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data, 2*, 150066. https://doi.org/10.1038/sdata.2015.66
- Mazviona, B., & Sølvsten, S. (2025). Can index insurance keep up with climate change? Rethinking historical data models. *Frontiers in Climate, 7*, 1649540. https://doi.org/10.3389/fclim.2025.1649540
- National Institute of Statistics of Rwanda (NISR). (2018–2025). *Seasonal Agricultural Survey* reports and microdata.
- Republic of Rwanda. (2021). Law N° 058/2021 of 13/10/2021 relating to the protection of personal data and privacy. *Official Gazette of the Republic of Rwanda*.
- Ryo, M. (2022). Explainable artificial intelligence and interpretable machine learning for agricultural data analysis. *Artificial Intelligence in Agriculture, 6*, 257–265.
- Stigler, M., & Lobell, D. (2024). Optimal index insurance and basis risk decomposition: An application to Kenya. *American Journal of Agricultural Economics, 106*(1), 306–329. https://doi.org/10.1111/ajae.12375
- Turvey, C. G., & McLaurin, M. K. (2012). Applicability of the Normalized Difference Vegetation Index (NDVI) in index-based crop insurance design. *Weather, Climate, and Society, 4*(4), 271–284. https://doi.org/10.1175/WCAS-D-11-00059.1